

Week 2: Learning, Backpropagation, and Optimization

Week 2 answers the missing question from Week 1: **how does loss become a parameter update?**

- Focus: training loop, gradient, chain rule, backpropagation, optimizer, diagnostics
- Output: students should be able to read loss curves and explain training failures
- Anchor sentence: backprop computes gradients; the optimizer uses them to update parameters

Where Week 2 Fits

Week 1 explained what an MLP can represent; Week 2 explains how its parameters are actually learned.

Roadmap

Week	Core theme	Student takeaway
1	MLP, activation, loss	Why nonlinear representation is needed
2	Gradient, backprop, optimizer	How loss becomes parameter updates
3	RNN/LSTM/GRU	Why sequence models need recurrence and memory
4	Attention	How token-to-token relevance is computed
5	Transformer + paper reading	How to read architecture and experiments

Today's Questions

Every slide should help answer one of these training questions.

- What does it mean for a model to **learn**?
- What direction does the gradient point to?
- Why is backpropagation a repeated chain rule?
- Why can the learning rate make training stable or unstable?
- How do SGD, Momentum, and Adam differ in practice?
- What evidence do loss curves give before we touch the architecture?

Training Loop at a Glance

Most deep learning training is a repeated loop of **forward** -> **loss** -> **backward** -> **update** -> **evaluate**.

Training Loop

Step	Code role	Meaning
1	<code>logits = model(x)</code>	forward pass: produce predictions
2	<code>loss = criterion(logits, y)</code>	turn mistakes into a scalar objective
3	<code>loss.backward()</code>	compute gradients for every parameter
4	<code>optimizer.step()</code>	move parameters using those gradients
5	validation logging	check whether the model generalizes

Objective Function and Update Rule

Learning means finding parameters that reduce the loss, not memorizing a magic API call.

Optimization View

$$\theta^* = \arg \min_{\theta} \mathcal{L}(f_{\theta}(X), y)$$
$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(f_{\theta}(X), y)$$

- **theta**: all trainable weights and biases
- **nabla_theta L**: gradient vector telling us how loss changes locally
- **eta**: learning rate, i.e. step size

Gradient Descent Intuition

The gradient points toward steepest ascent, so we move in the opposite direction to reduce the loss.

Learning Rate Trajectories

Learning Rate Changes the Optimization Trajectory



- Too small: progress is safe but painfully slow
- Reasonable: reaches the valley efficiently
- Too large: overshoots and can bounce or diverge

Mini-batch Training

We usually update parameters using a mini-batch estimate of the full gradient.

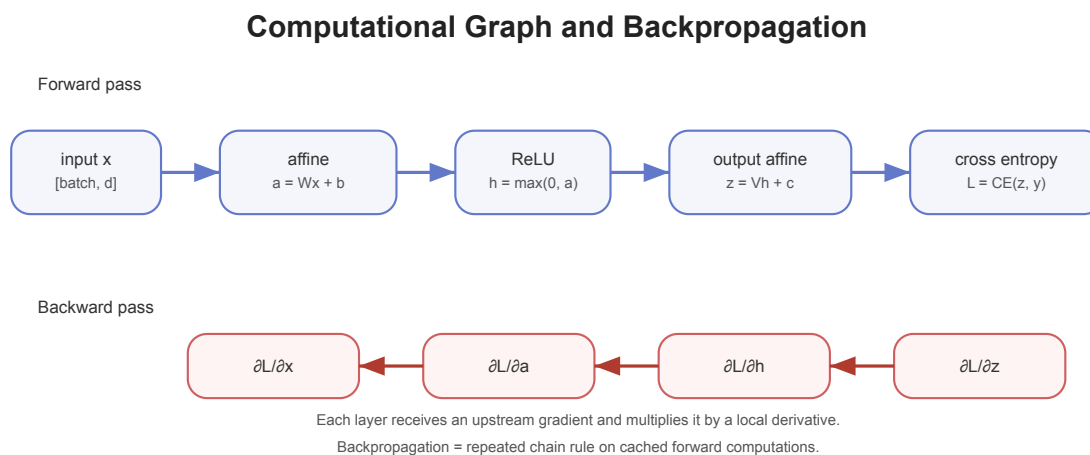
Batch Choices

Method	Gradient source	Practical trade-off
Full-batch GD	entire dataset	stable direction, slow updates
Mini-batch SGD	small subset	noisy direction, frequent updates
One-sample SGD	single example	very noisy, sometimes useful for intuition

Computational Graph

Backpropagation is easiest to understand if the model is seen as a graph of intermediate computations.

Computational Graph



- Forward pass stores intermediate values needed later
- Backward pass sends an upstream gradient backward through each node
- Each layer combines the incoming gradient with its own local derivative

Chain Rule on a Tiny Graph

A complicated network still reduces to many simple local derivatives multiplied together.

Scalar Example

$$\begin{aligned}a &= wx + b \\h &= \text{ReLU}(a) \\L &= h^2 \\ \frac{\partial L}{\partial h} &= 2h \\ \frac{\partial h}{\partial a} &= \mathbf{1}[a > 0] \\ \frac{\partial a}{\partial w} &= x \\ \therefore \frac{\partial L}{\partial w} &= \frac{\partial L}{\partial h} \frac{\partial h}{\partial a} \frac{\partial a}{\partial w}\end{aligned}$$

Backpropagation in Layer Language

The backward pass asks each layer one question: **given the gradient from above, what gradient should I pass backward?**

- The output layer receives the loss gradient first
- A hidden layer multiplies that upstream signal by its local Jacobian
- The process repeats until every parameter receives its gradient
- Backpropagation is therefore gradient computation, not the update itself

Affine Layer Gradient Shapes

For a linear layer, gradient computation is mostly disciplined shape bookkeeping.

Layer Derivatives

$$X \in \mathbb{R}^{n \times d}, W \in \mathbb{R}^{d \times h}, b \in \mathbb{R}^h$$

$$A = XW + b \in \mathbb{R}^{n \times h}$$

$$\frac{\partial L}{\partial A} \in \mathbb{R}^{n \times h}$$

$$\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial A} \in \mathbb{R}^{d \times h}$$

$$\frac{\partial L}{\partial b} = \sum_{i=1}^n \frac{\partial L}{\partial A_i} \in \mathbb{R}^h$$

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial A} W^\top \in \mathbb{R}^{n \times d}$$

Two-Layer MLP Backward Flow

Even for an MLP, the backward pass is still just repeated local rules.

MLP Example

$$\begin{aligned}A_1 &= XW_1 + b_1 \\H_1 &= \text{ReLU}(A_1) \\Z &= H_1W_2 + b_2 \\L &= \text{CE}(Z, y) \\\delta_2 &= \frac{\partial L}{\partial Z} \\\frac{\partial L}{\partial W_2} &= H_1^\top \delta_2 \\\delta_1 &= (\delta_2 W_2^\top) \odot \mathbf{1}[A_1 > 0] \\\frac{\partial L}{\partial W_1} &= X^\top \delta_1\end{aligned}$$

PyTorch: backward and step

PyTorch hides the symbolic differentiation, but the conceptual roles are still visible in code.

Minimal Core

```
logits = model(X_batch)          # forward pass
loss = criterion(logits, y_batch)

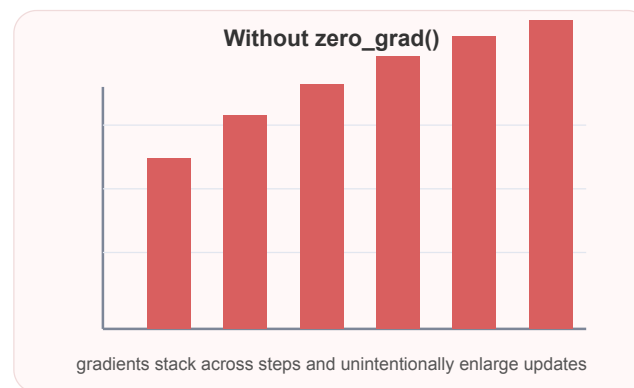
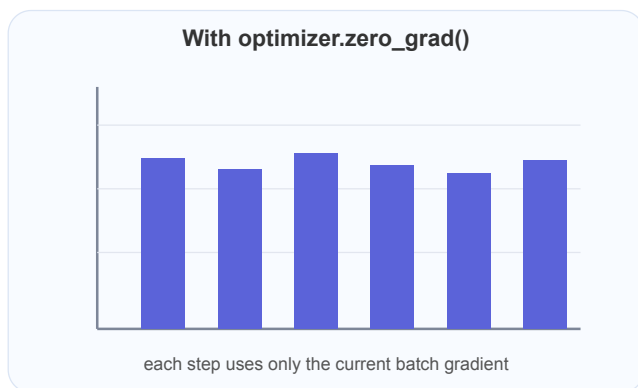
optimizer.zero_grad()           # clear old gradients
loss.backward()                  # compute dL/dtheta for all parameters
optimizer.step()                 # update theta using the optimizer rule
```

Why `optimizer.zero_grad()` Matters

If we forget to clear old gradients, PyTorch accumulates them across steps.

Gradient Accumulation

Forgetting `zero_grad()` Causes Gradient Accumulation



- Expected behavior: each step uses gradients from the current batch
- Without `zero_grad()`: new gradients are added on top of old ones
- Symptom: updates become unintentionally larger and unstable

Optimizer Family

Optimizers differ in how they convert the same gradient information into actual parameter motion.

Common Optimizers

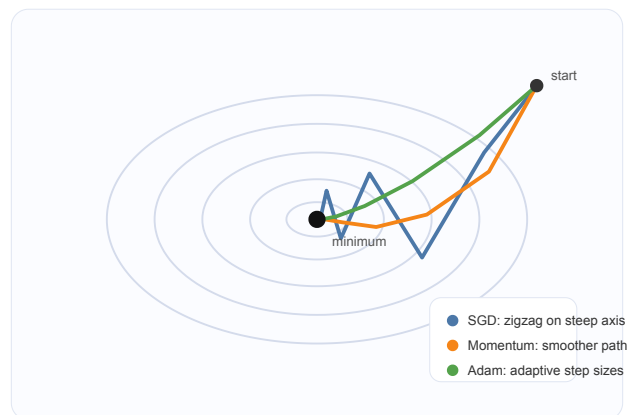
Optimizer	Update idea	Classroom interpretation
Gradient Descent	use full gradient	clean concept, slow at scale
SGD	use mini-batch gradient	noisy but standard
Momentum	accumulate velocity	smooths zigzags and keeps direction
Adam	adaptive per-parameter scaling + momentum	usually fast and convenient

Optimizer Trajectories

The same loss surface can produce very different motion depending on the optimizer.

Optimizer Paths

Optimizer Trajectories on the Same Loss Surface



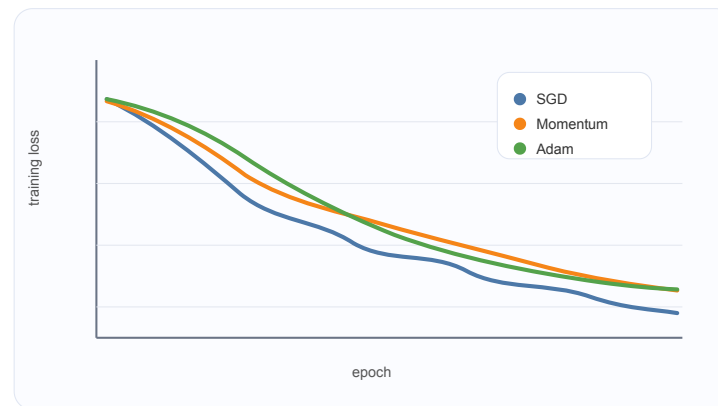
- SGD may zigzag strongly on steep directions
- Momentum keeps a running direction and damps oscillation
- Adam rescales updates per parameter and often reaches a good region quickly

Optimizer Comparison by Loss Curve

Beyond parameter paths, we often compare optimizers through the stability and speed of loss reduction.

Optimizer Loss Curves

Illustrative Optimizer Loss Curves

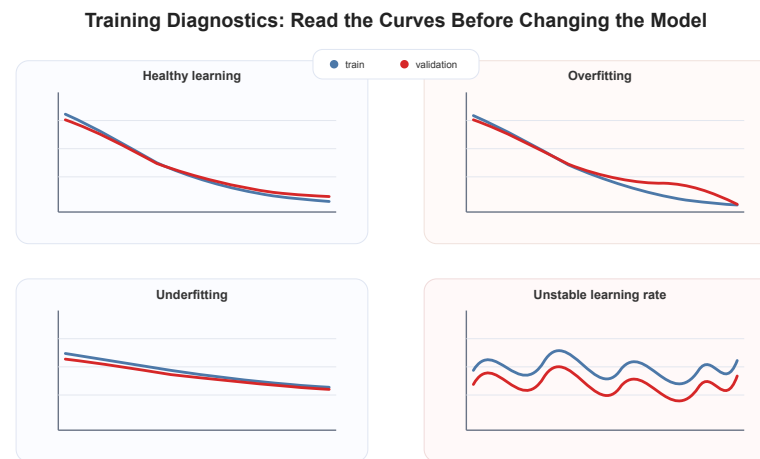


- A faster initial drop is not the only criterion
- We also care about oscillation, final validation quality, and reproducibility
- A low training loss with poor validation can still be the wrong choice

Training Diagnostics

Before redesigning the architecture, first read the curves and classify the failure mode.

Training Diagnostics



Reading Failure Patterns

Good debugging starts with a careful verbal interpretation of train and validation behavior.

Curve Patterns

Case	Train loss	Validation loss	Likely interpretation
Healthy learning	decreases	decreases then stabilizes	model is learning and generalizing
Overfitting	keeps decreasing	starts increasing	memorizing training details
Underfitting	stays relatively high	stays relatively high	capacity or optimization is insufficient
Unstable training	oscillates or explodes	oscillates or explodes	learning rate / gradient issue

Minimal PyTorch Training Loop

Week 2 practice should make students write the loop themselves, not fill blanks in a template.

Template

```
model = MLP()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(num_epochs):
    model.train()
    for X_batch, y_batch in train_loader:
        logits = model(X_batch)
        loss = criterion(logits, y_batch)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Validation Loop and Logging

Training without logs is not an experiment; it is just execution.

Evaluation Block

```
model.eval()
val_loss = 0.0
val_correct = 0

with torch.no_grad():
    for X_batch, y_batch in val_loader:
        logits = model(X_batch)
        loss = criterion(logits, y_batch)
        val_loss += loss.item()
        val_correct += (logits.argmax(dim=1) == y_batch).sum().item()
```

- Log at least: train loss, validation loss, train metric, validation metric
- Keep the x-axis consistent: epoch or optimization step
- Loss curves belong in every Week 2 submission

Week 2 Experiment Protocol

The assignment is a controlled optimizer study, not a random hyperparameter search.

- Fix one model and one dataset
- Compare at least: SGD, SGD + Momentum, Adam
- Test at least three learning rates
- Save train/validation curves and final metrics
- Include one failed setting and explain why it failed

What to Submit

The deliverable should read like a small research note, not just a notebook dump.

Submission

Artifact	Why it matters
Loss curve image	shows optimization stability and overfitting behavior
Final train/validation metric table	enables direct comparison
Short explanation of the most stable optimizer	forces interpretation
One failed configuration + hypothesis	teaches debugging language
Code or notebook	ensures reproducibility

Checkpoint

By the end of Week 2, students should be able to explain these sentences aloud.

- Backpropagation computes gradients; it does not directly update parameters.
- The optimizer uses computed gradients to change parameters.
- The learning rate controls update size and can stabilize or destabilize training.
- Momentum changes the path of optimization without changing the model class.
- Loss curves are the first piece of evidence when debugging training.

Next Bridge: From Optimization to Sequence Models

Once students understand how parameters are learned, Week 3 can focus on why ordinary MLPs are not enough for sequences.

- Week 2 asked: how does loss update parameters?
- Week 3 will ask: why does sequence data require memory over time?
- The training loop stays; the architecture changes
- This continuity is important when students later read RNN or Transformer papers

